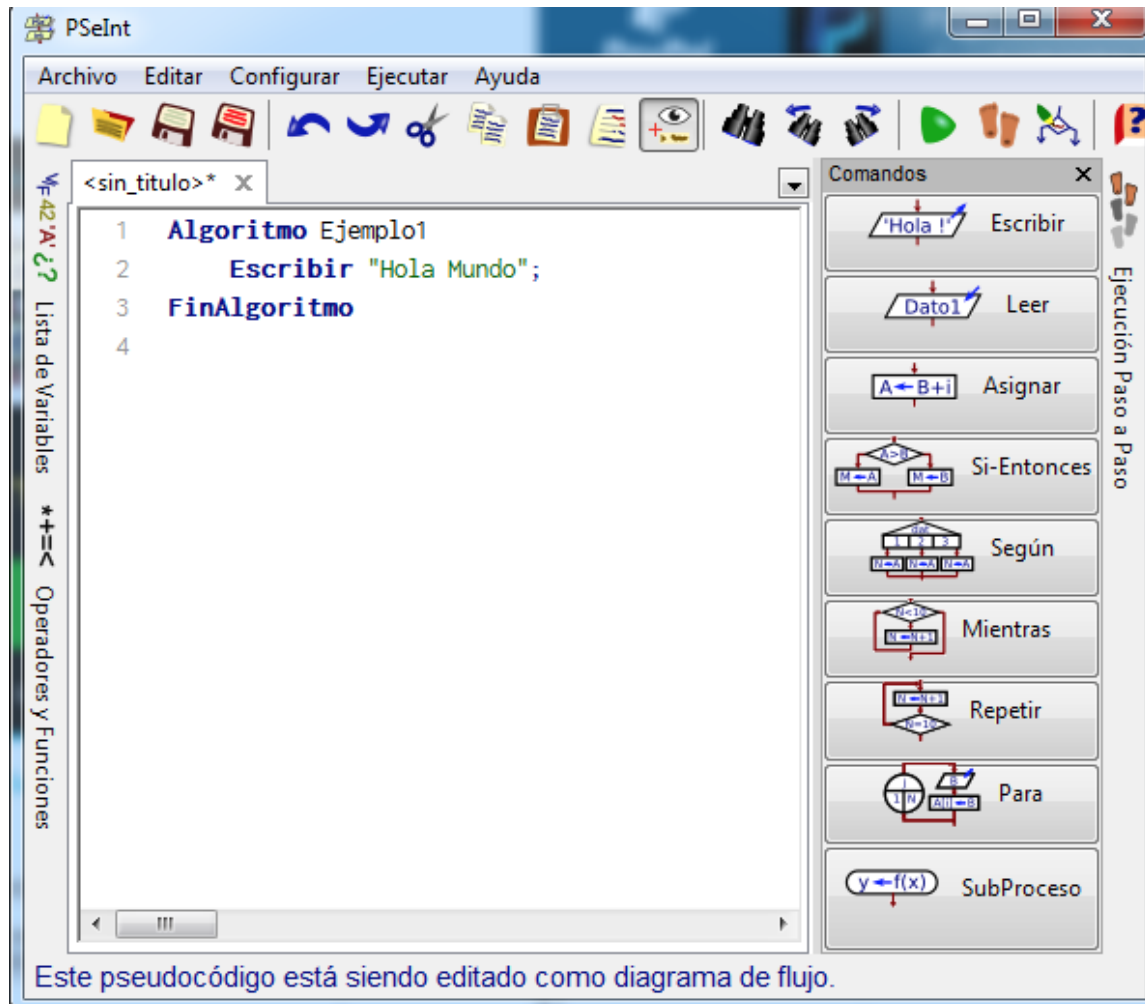


PSEUDOCÓDIGO

Programación



PARTE I

ÍNDICE

INTRODUCCIÓN A LA PROGRAMACIÓN.....	3
<i>Comprensión del Problema</i>	<i>4</i>
<i>Encontrar una solución</i>	<i>4</i>
<i>Enunciar un Algoritmo</i>	<i>5</i>
ALGORITMO.....	5
CARACTERÍSTICAS DE UN ALGORITMO COMPUTACIONAL	6
DIAGRAMA	6
<i>DFD - DIAGRAMAS DE FLUJO DE DATOS.....</i>	<i>7</i>
<i>PSEUDOCÓDIGO</i>	<i>8</i>
ESTRUCTURA DE UN PROGRAMA.....	9
EL CÓDIGO.....	9
<i>SINTAXIS.....</i>	<i>10</i>
<i>INSTRUCCIONES</i>	<i>11</i>
<i>SENTENCIAS.....</i>	<i>11</i>
ESTRUCTURA DE DATOS.....	12
<i>Constantes.....</i>	<i>12</i>
<i>Variables.....</i>	<i>12</i>
EXPRESIONES	14
OPERADORES.....	15
<i>ARITMÉTICOS.....</i>	<i>15</i>
<i>RELACIONALES.....</i>	<i>15</i>
<i>LÓGICOS.....</i>	<i>16</i>
<i>CONCATENACIÓN.....</i>	<i>17</i>
ORDEN DE EVALUACIÓN DE LOS OPERADORES	17
COMENZANDO A PROGRAMAR	20
ALGORITMO DE SUMA.....	20
ALGORITMO DEL TRIANGULO.....	22

INTRODUCCIÓN A LA PROGRAMACIÓN

La razón principal por la que una persona utiliza un ordenador es para **resolver problemas** (en el sentido más general de la palabra), o en otras palabras, procesar una información para obtener un resultado a partir de unos datos de entrada.

Los ordenadores resuelven los problemas mediante la **utilización de programas escritos por los programadores**. Los programas de ordenador no son entonces más que métodos para resolver problemas. Por ello, para escribir un programa, lo primero es que el programador sepa resolver el problema que estamos tratando.

El programador debe identificar cuáles son los datos de entrada y a partir de ellos obtener los datos de salida, es decir, la solución, a la que se llegará por medio del procesamiento de la información que se realizará mediante la utilización de un método para resolver el problema que denominaremos **algoritmo**.

Se define como **PROGRAMACIÓN** al proceso de dotar al ordenador de un método para resolver un problema tipo y entregar un resultado.

Se dice también que un programa está compuesto de dos partes:

- El **código**, que consiste en el conjunto de acciones a ejecutar, en programación a cada acción básica se la denomina genéricamente como **instrucción**. Cada instrucción podrá estar compuesta de un conjunto de elementos los cuales estudiaremos más adelante.
- La **estructura de datos** que consiste en la disposición de memoria elegida para almacenar los datos necesarios para resolver el problema, ya sea los parámetros o datos de entrada, los resultados intermedios o datos de proceso y la información resultante o datos de salida. A estos datos en programación se los denomina genéricamente como **variables**.

El ordenador es una máquina que por sí sola no puede hacer nada, necesita ser programada, es decir, introducir a su memoria instrucciones que la guíen en lo que tiene que hacer. El ordenador solamente es capaz de realizar tareas muy básicas, por lo tanto, hay que definir claramente cuál será el proceso a ejecutar para hacer tareas complejas mediante pasos elementales.

Para comprenderlo más organizadamente, podemos decir que este proceso de programación se divide en las siguientes tareas:

1. ***Comprensión del problema.***
2. ***Idear una solución que sea lo más general posible.***
3. ***Enunciar un algoritmo de resolución.***
4. ***Codificarlo en un lenguaje de programación.***
5. ***Implementarlo***, o, dicho de otra manera, hacer que la máquina pueda traducirlo a su propio código binario.
6. En forma paralela a los pasos anteriores, preparar un ***lote de prueba***.
7. ***Ejecutar el programa y validarlo mediante el lote de prueba***, si se encuentran errores lógicos, corregirlos, o sea, volver al punto 3 o 4 según el nivel de error.

Comprensión del Problema

En esta etapa, se debe confeccionar el enunciado del problema, el cual requiere una definición clara y precisa. Es importante que se conozca exactamente cuál es el objetivo que debe resolverse o, dicho de otra manera, lo que se desea que realice el ordenador.

Quizás quien enuncie el problema no sepa definir claramente los objetivos, así que el analista deberá interrogarlo hasta que el objetivo a resolver quede perfectamente claro, mientras esto no se logre completamente, no tiene mucho sentido continuar con la siguiente etapa.

Encontrar una solución

Normalmente esta etapa comienza en paralelo a la lectura del enunciado, cuando se va comprendiendo el enunciado y por experiencia o analogía se van ideando las acciones que se deben aplicar para llegar al resultado esperado.

La solución debe ser lo más amplia o general posible.

Es necesario en esta etapa tener en claro:

- Cuáles son los datos que se requieren para definir el caso particular, los denominados datos de entrada.
- Cuál es el conjunto de resultados que se desea obtener, o sea, la información de salida.
- Los métodos y fórmulas que se necesitan para procesar los datos.

Enunciar un Algoritmo

Se debe plantear el método de resolución elegido mediante una construcción matemática denominada algoritmo.

ALGORITMO

Por algoritmo entendemos un conjunto ordenado y finito de operaciones (o pasos) que permiten resolver un problema que además cumplen las siguientes características:

- Tiene un número finito de pasos
- Acaba en un tiempo finito. Si no acabase nunca, no se resolvería el problema.
- Todas las operaciones deben estar definidas de forma precisa y sin ambigüedad.
- Puede tener varios datos de entrada y como mínimo un dato de salida.

Un claro ejemplo de algoritmo es una receta de cocina, donde tenemos unos pasos que hay que seguir en un orden y deben de estar bien definidos, tiene un tiempo finito y tiene unos datos de entrada (ingredientes) y una salida (el plato).

Por ejemplo, el algoritmo para freír un huevo podría ser el siguiente:

Datos de entrada: Huevo, aceite, sartén, fuego.

Datos de salida: huevo frito.

Procedimiento:

1. Poner el aceite en la sartén.
2. Poner la sartén al fuego.
3. Cuando el aceite esté caliente, cascar el huevo e introducirlo.
4. Cubrir el huevo de aceite.
5. Cuando el huevo esté hecho, retirarlo.

CARACTERÍSTICAS DE UN ALGORITMO COMPUTACIONAL

La codificación de un algoritmo en un ordenador se denomina programa .

Para poder enunciar un algoritmo que se pueda codificar en un lenguaje de programación, debemos asumir que quién interpretará el algoritmo (la CPU) podrá realizar solamente acciones muy básicas.

Nos concentraremos solamente en aquellas acciones que son comunes a cualquier lenguaje de programación y no contemplaremos ninguna acción que sea exclusiva de un lenguaje. Así podemos determinar que las acciones que serán permitidas en un algoritmo son las siguientes:

- Introducir un valor en una variable desde un dispositivo de entrada (el teclado, por ejemplo)
- Mostrar en un dispositivo de salida un valor ya sea de una variable o un valor literal (en la pantalla, por ejemplo)
- Almacenar un valor en una variable como resultado de una expresión (como se denomina un cálculo en el lenguaje de la programación)
- Bifurcar el flujo de ejecución entre dos alternativas, mediante una condición evaluada.
- Repetir parte del flujo normal de proceso de una manera condicional.
- Iterar, o sea repetir parte del flujo un número determinado de veces.
- Seleccionar un único flujo de proceso a ejecutar de un conjunto de flujos posibles.

DIAGRAMA

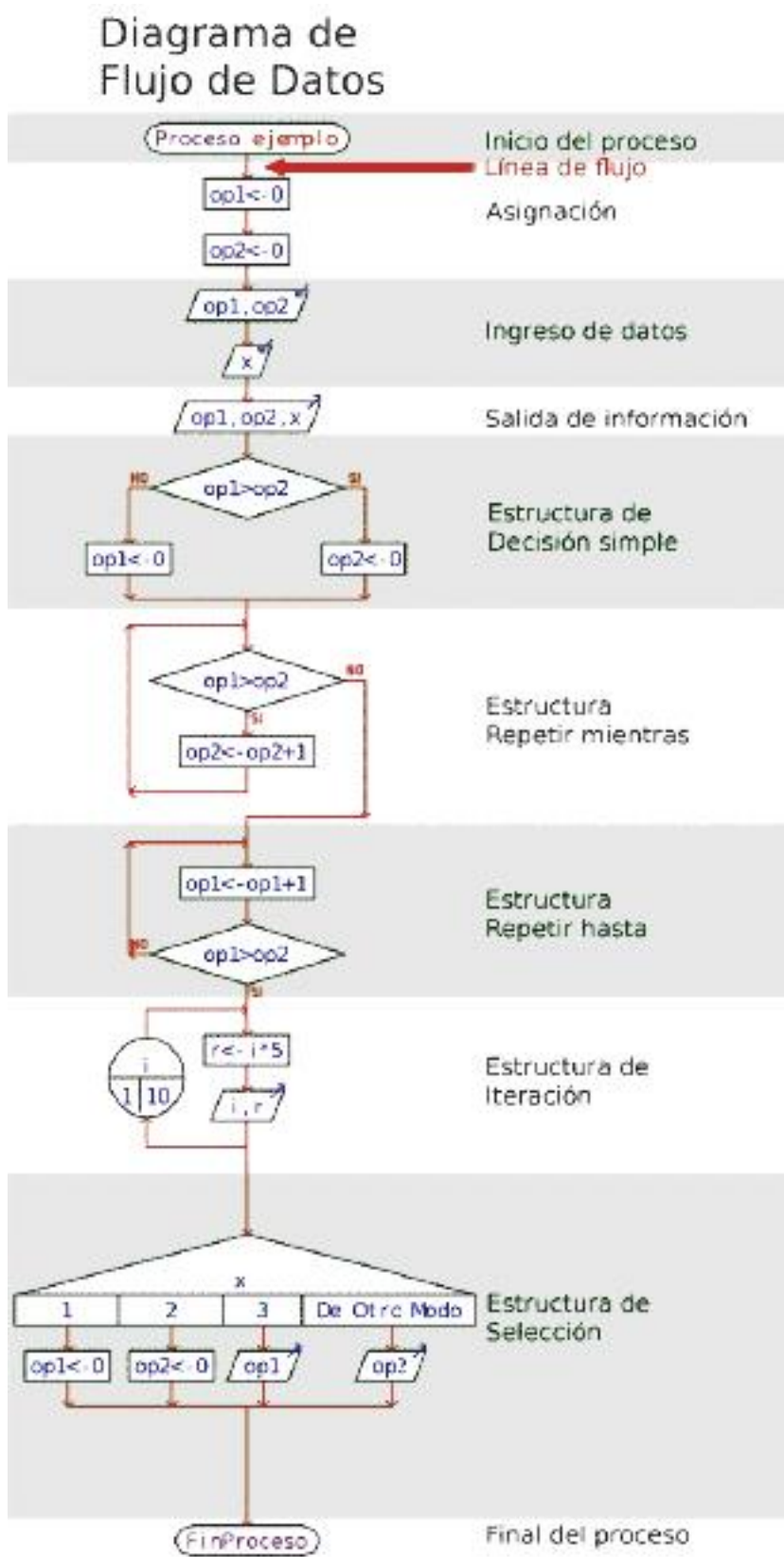
Un diagrama es una **representación gráfica** de una proposición, de la resolución de un problema, de las relaciones entre las diferentes partes o elementos de un conjunto o sistema.

Existen muchísimos tipos de diagramas cada uno es aplicable de acuerdo a la información que se quiera representar, así por ejemplo en teoría de conjuntos tenemos los diagramas de Venn, en bases de datos se utilizan los diagramas de Entidad-Relación. En nuestro caso, la programación, se utilizan principalmente los **diagramas de Flujo de Datos**.

DFD - DIAGRAMAS DE FLUJO DE DATOS

Los diagramas de flujo se utilizan en muchos campos del conocimiento fuera de la programación, por ejemplo, en el ámbito empresarial, en medicina, etc. En los diagramas de flujo de datos (abreviado DFD), el flujo precisamente está indicado por una línea descendente que indica la secuencia de ejecución de las instrucciones.

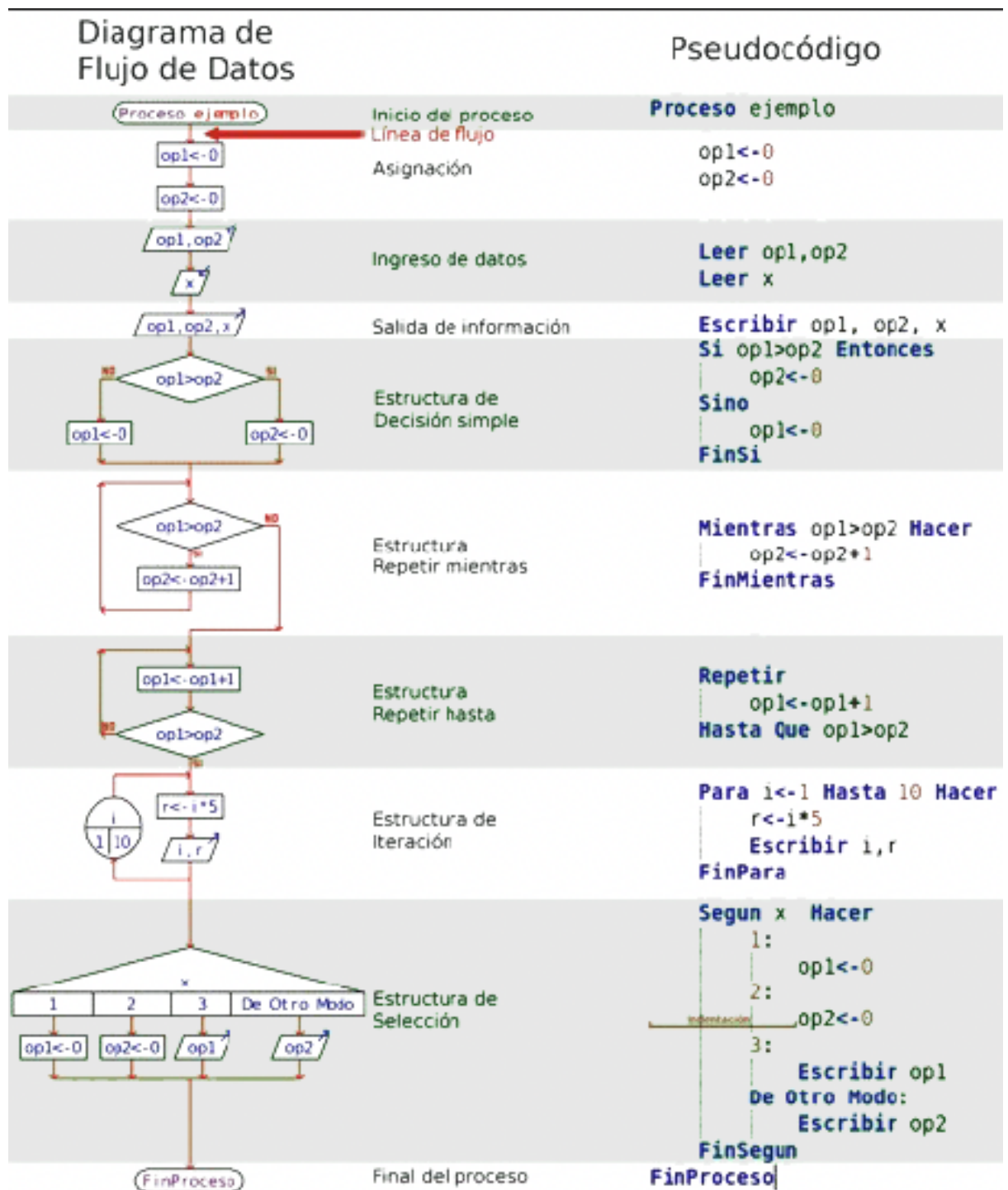
En la ilustración podemos observar todos los elementos del diagrama de flujo que maneja el programa PseInt.



PSEUDOCÓDIGO

El Pseudocódigo es otra forma de representar un algoritmo, en este caso utilizando una notación similar a un lenguaje de programación estándar, pero cuyas palabras claves están escritas en el lenguaje castellano. Existe una correspondencia entre los elementos del DFD y el Pseudocódigo como se puede apreciar en la ilustración.

La idea es que se pueda ver el programa desde el punto de vista del programador el cual está acostumbrado a leer código fuente.



ESTRUCTURA DE UN PROGRAMA

Habíamos mencionado previamente que en un programa encontramos dos partes, el código y la estructura de datos.

EL CÓDIGO

El código podemos verlo como un texto que describe la receta que se debe aplicar para realizar una tarea compleja de procesamiento de datos. Pero... ¿Con qué objetivo se define este texto?

La idea es que la computadora puede ejecutar programas que traducen texto generado por el usuario (denominado código fuente) a operaciones en código máquina (denominado código objeto) y puede ejecutar cada acción descrita en cada oración. Estos programas traductores a código máquina se denominan genéricamente como programas intérpretes. El único problema es que los lenguajes naturales son poco precisos al momento de definir tareas, ya que nos encontramos con que podemos utilizar un conjunto de sinónimos para representar lo mismo o incluso una misma palabra presenta ambivalencias, es decir en un contexto significa una cosa y en otro contexto otra cosa. Para no hacer demasiado complejo al programa intérprete, estos reconocen un lenguaje muy acotado (que se basa en su amplia mayoría en el idioma inglés) resumiendo el vocabulario normal a unas pocas palabras con un significado preciso, incluso muchas de esas palabras son inexistentes en el lenguaje natural y solo tienen sentido en la programación. El objetivo es hacer que el texto que podamos armar no sea difícil de entender y traducir por no tener ninguna ambivalencia en su significado. Como todo texto, un programa no es más que un conjunto de oraciones, en este caso un programa se compone de oraciones de tipo declarativas e imperativas.

Retomando la idea entonces, cuando programamos no hacemos más que *describir paso por paso y con oraciones muy simples que debe hacer o tener en consideración la computadora para aplicar un algoritmo.*

Como todo lenguaje, la oración está sujeta a leyes sintácticas que en este caso son muy estrictas. Por eso la primera tarea que el programa traductor hace, luego de que nosotros codifiquemos el programa es realizar un análisis sintáctico de todo el texto o de cada oración indicando todos los errores sintácticos detectados para que los corrijamos. Solo cuando el programa se haya libre de errores sintácticos se puede comenzar con la traducción a código máquina y posterior ejecución.

SINTAXIS

Las reglas sintácticas son más bien pocas, pero muy restrictivas. Cada lenguaje tiene sus propias restricciones. En el caso del pseudocódigo podemos mencionar las siguientes reglas:

Se define un conjunto de palabras reservadas. Estas palabras solo pueden utilizarse en un único sentido, el que se asigne como definición en el lenguaje.

Las palabras reservadas y su significado son:

PALABRA RESERVADA	SIGNIFICADO
Leer <v>	Ingresar un conjunto de datos desde el teclado hacia variables de memoria, se menciona al teclado como generalización de cualquier periférico de entrada.
Escribir <e>	Escribir por la pantalla un conjunto de datos, se menciona la pantalla como una generalización de dispositivo de salida.
<v> <- <e>	Asignación de un dato a una variable de memoria
Si <e> Entonces SiNo FinSi	Bifurcar el flujo normal de operación entre dos flujos alternativos.
Mientras <e> Hacer finMientras	Repetir una secuencia de instrucciones mientras se cumpla una condición
Repetir Hasta que <e>	Repetir una secuencia de instrucciones una cantidad fija de veces.
Según <v> hacer <k0>: <k1>: ... <kn>: De Otro Modo: finSegún	Bifurcar el flujo normal de operación entre varios flujos alternativos.
Proceso <n> finProceso	Delimitador de puntos de entrada y de salida de un proceso cuyo nombre es indicado por <n>.

Como observamos en la tabla de palabras reservadas, utilizamos unos símbolos para indicar elementos obligatorios, vamos a definirlos también:

- <v> indica que se debe especificar un dato variable (o una variable a secas) el cual representa un espacio en la memoria del equipo donde se almacenará un valor.
- <k> indica que se deberá utilizar un valor constante, o sea que tiene que estar definido literalmente.

- `<e>` indica una expresión, o sea, el resultado de un cálculo entre uno o varios datos. Una expresión además de un resultado también puede ser un valor constante o variable.
- `<n>` palabra definida por el usuario (p.ej. un nombre de variable o de proceso).

Otros elementos que forman parte del código son los operadores, los delimitadores y las funciones. Más adelante hablaremos de ellos.

INSTRUCCIONES

Son oraciones imperativas cuya estructura es siempre: “hacer tal cosa con tales datos”. El núcleo de la oración es el verbo que indica la orden que queremos que se ejecute, lo llamaremos precisamente orden o comando. Los verbos tienen un solo modo, el infinitivo.

En los lenguajes de programación hay multitud de comandos, pero en el pseudocódigo vamos a enfocarnos solamente en tres ya mencionados:

- Leer
- Escribir
- `<-` (asignar)

SENTENCIAS

Son oraciones declarativas que sirven para dividir la secuencia de ejecución en bloques que incluyan instrucciones u otras sentencias (a estas se las denomina sentencias anidadas) que se ejecutaran en forma grupal, delimitando donde comienza y donde termina este bloque. Por comodidad denominaremos a este conjunto como estructura. En este caso, las existencias de estas sentencias definen a la metodología de programación como programación estructurada.

Las sentencias son:

- Proceso ... / finProceso
- Si ... Entonces / Sino / finSi
- Mientras ... Hacer / finMientras
- Repetir / Hasta que ...
- Para ... `<-` ... Hasta ... Hacer / finPara

ESTRUCTURA DE DATOS

La estructura de datos está formada por el conjunto de datos variables que requiere el programa.

Entendemos por objeto de un programa todo aquello que pueda ser manipulado por las instrucciones. En ellos se almacenarán tanto los datos de entrada como los de salida (resultados).

Sus atributos son:

- Nombre: el identificador del objeto.
- Tipo: conjunto de valores que puede tomar.
- Valor: elemento del tipo que se le asigna.

Constantes

Son objetos cuyo valor permanece invariable a lo largo de la ejecución de un programa. Una constante es la denominación de un valor concreto, de tal forma que se utiliza su nombre cada vez que se necesita referenciarlo.

Por ejemplo: $\pi = 3.14.1592$ e $e = 2.718281$

También son utilizadas las constantes para facilitar la modificabilidad de los programas, es decir para hacer más independientes ciertos datos del programa. Por ejemplo, supongamos un programa en el que cada vez que se calcula un importe al que se debe sumar el IVA utilizáramos siempre el valor 0.21, en caso de variar este índice tendríamos que ir buscando a lo largo del programa y modificando dicho valor, mientras que si le damos nombre y le asignamos un valor, podremos modificar dicho valor con mucha más facilidad.

Variables

Son objetos cuyo valor puede ser modificado a lo largo de la ejecución de un programa.

Por ejemplo: una variable para calcular el área de una circunferencia determinada, una variable para calcular una factura, etc.

Respecto al nombre, cada variable debe poseer un nombre único, éste debe comenzar por una letra (vocal o consonante) y luego puede continuar con un conjunto de letras o números. Queda prohibido como parte del nombre cualquier símbolo y tampoco se admiten espacios.

Respecto a los tipos primitivos de datos, en pseudocódigo, existen tan solo tres:

- El tipo **lógico** o booleano, el cual representa un dato que puede tener sólo dos posibles valores: verdadero o falso.
- El tipo **numérico**, el cual representa un dato que puede operarse matemáticamente, por ejemplo, una medida, el resultado de un cálculo, etc. este valor debe pertenecer al conjunto de números reales.
- El tipo **alfanumérico**, el cual representa un dato que no es matemático, por ejemplo, un nombre, una denominación, un código, etc. a los alfanuméricos también se los conoce como strings o cadenas de texto.

EXPRESIONES

Ya habíamos mencionado que una expresión consiste en el resultado de un cálculo, pero debemos ser un poco más extensos en su definición ya que involucra otros elementos de la programación que no hemos mencionado y también por su importancia ya que es una de las construcciones más utilizadas en los programas.

Las expresiones según el resultado que produzcan se clasifican en:

- **Numéricas:** Son las que producen resultados de tipo numérico. Se construyen mediante los operadores aritméticos.

Por ejemplo:

$\pi * \text{sqr}(x)$ o $(2*x)/3$

- **Alfanuméricas:** Son las que producen resultados de tipo alfanumérico. Se construyen mediante operadores alfanuméricos.

Por ejemplo:

"Don " + "José"

- **Booleanas o lógicas:** Son las que producen resultados de tipo Verdadero o Falso. Se construyen mediante los operadores relacionales y lógicos.

Por ejemplo:

$a < 0$ o $(a > 1)$ and $(b < 5)$

Una expresión está compuesta por datos afectados por operadores y/o funciones.

Un operador es un símbolo que representa una operación entre datos.

Algunos son muy conocidos como por ejemplo + y - (la suma y la resta), pero hay muchos más.

Una función es un proceso definido que se hace con uno o varios datos y del cual se obtiene un único resultado. Por ejemplo las funciones trigonométricas seno(), coseno(), etc.

OPERADORES

Los operadores pueden ser de los siguientes tipos: matemáticos o aritméticos, relacionales, lógicos y por último un operador llamado concatenación.

ARITMÉTICOS

Operadores matemáticos son la suma (+), la resta (-), la división (/), la multiplicación (*), la potenciación (^) y el módulo o resto de la división (%). El resultado de una operación matemática es un tipo numérico.

OPERADOR	DEFINICIÓN
+	Suma
-	Resta
*	Multiplicación
^	Potencia
/	División
%	Resto de la división

Ejemplos:

EXPRESIÓN	RESULTADO
3+ 5 - 2	6
24 % 3	0
8 * 3 - 7 / 2	20.5

RELACIONALES

Operadores relacionales son: el mayor (>), menor (<) e igual (=) y sus operaciones complementarias: menor o igual (<=), mayor o igual (>=) y distinto (<>) respectivamente. El resultado es un tipo lógico ya que nos dice si la relación es verdadera o falsa.

OPERADOR	DEFINICIÓN
<	Menor que
>	Mayor que
=	Igual que
>=	Mayor o igual que
<=	Menor o igual que
<>	Distinto que

Ejemplos:

EXPRESIÓN	RESULTADO
'A' < 'B'	Verdadero
1 < 6	Verdadero
10 < 2	Falso
9 = (3 * 3)	Verdadero
3 <> 2	Verdadero

LÓGICOS

Operadores lógicos: se utilizan para unir subexpresiones de tipo lógico, ellas son la conjunción AND (&) la disyunción OR (|) y el complemento NOT (~), el resultado es un tipo lógico siguiendo las reglas de las tablas de verdad del álgebra de Boole.

OPERADOR	DEFINICIÓN
No	Negación
Y	Conjunción
O	Disyunción

El comportamiento de un operador lógico se define mediante su correspondiente tabla de verdad, en ella se muestra el resultado que produce la aplicación de un determinado operador a uno o dos valores lógicos. Las operaciones lógicas más usuales son:

- NO lógico (NOT) o negación:

A	NOT A
V	F
F	V

El operador NOT invierte el valor: Si es verdadero (V) devuelve falso (F), y viceversa.

- O lógica (OR) o disyunción:

A	B	A OR B
V	V	V
V	F	V
F	V	V
F	F	F

El operador OR devuelve verdadero (V) si alguno de los dos valores es verdadero. De lo contrario, devuelve Falso (F).

- Y lógica (AND) o conjunción:

A	B	A AND B
V	V	V
V	F	F
F	V	F
F	F	F

El operador AND devuelve Verdadero (V) solo si ambos valores son verdaderos. En cualquier otro caso devuelve Falso (F).

Ejemplos (Suponiendo que $a < b$):

EXPRESIÓN	RESULTADO
$9 = (3 * 3)$	Verdadero
$3 <> 2$	Verdadero
$9 = (3*3) \text{ Y } 3 <> 2$	Verdadero Y Verdadero = Verdadero
$3 > 2 \text{ Y } 'B' < 'A'$	Verdadero Y Falso = Falso
$3 > 2 \text{ O } 'B' < 'A'$	Verdadero O Falso = Verdadero
No ('A' < 'B')	No Verdadero = Falso
$5 > 1 \text{ Y No } ('B' < 'A')$	Verdadero Y No Falso = Verdadero

CONCATENACIÓN

Operador de concatenación: es el único operador que permite operar entre si dos datos de tipo alfanumérico, el resultado es una cadena de texto que contiene las dos cadenas unidas una continuación de la otra.

Ejemplos:

EXPRESIÓN	RESULTADO
"Ana" + "López"	Ana López
"saca" + "puntas"	sacapuntas

Casi todas las operaciones requieren que existan dos operandos, uno a derecha y otro a izquierda, por eso es que a la mayoría se los clasifica cómo de tipo binario. Con las siguientes excepciones:

- El operador NOT (~) solo admite un operando, por lo tanto se lo clasifica cómo operador monario.
- El operador menos (-) se puede utilizar para realizar una sustracción, con lo cual es del tipo binario, pero también se lo puede utilizar como operador de cambio de signo, actuando en este caso como monario.

ORDEN DE EVALUACIÓN DE LOS OPERADORES

Una expresión cuando es muy compleja puede analizarse dividiéndola en varias subexpresiones más simples, ya que en realidad la computadora va a operar con pares de datos o a veces con datos individuales. La lógica indicaría que la computadora irá ejecutando las subexpresiones de izquierda a derecha, pero no es así, ya que algunas

operaciones tienen mayor prioridad que otras, se establecen entonces reglas de precedencia que indican que operador va a tener mayor prioridad de resolución sobre otro, y si ambos tuvieran la misma prioridad, se resuelven de izquierda a derecha.

A la hora de resolver una expresión, el orden a seguir es el siguiente:

1. Paréntesis.
2. Potencia ^
3. Multiplicación, división y resto * / %
4. Sumas y restas + -
5. Concatenación +
6. Relacionales < <= > >= etc.
7. Negación NOT
8. Conjunción AND
9. Disyunción OR

La evaluación de operadores de igual orden se realiza de izquierda a derecha.

Veamos un ejemplo tomando la fórmula de cálculo de la distancia recorrida por un proyectil en tiro vertical. En matemática se expresaría como:

$$d = v_i \cdot t - \frac{1}{2} g \cdot t^2$$

donde d es la distancia recorrida (la incógnita), v_i es la velocidad inicial del proyectil, t es el tiempo que transcurrió y g es la aceleración de la gravedad.

Esto en pseudocódigo se expresaría como:

$$d \leftarrow v_i * t - 1 / 2 * g * t ^ 2$$

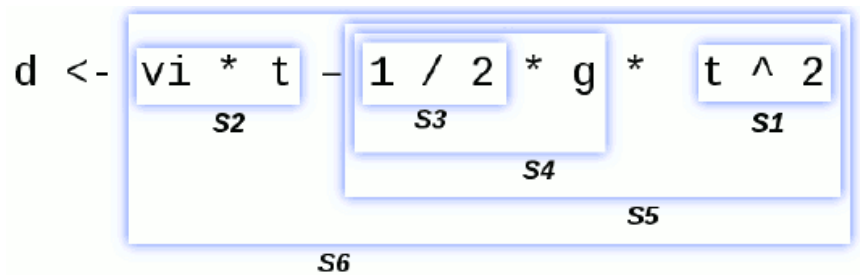
definamos como lote de prueba los siguientes valores: $v_i \leftarrow 100$

$t \leftarrow 10$

$g \leftarrow 9.8$

Analicemos por parte, primero d es la variable de destino, no forma parte de la expresión y el símbolo < es la instrucción que indica la acción de almacenamiento en la memoria.

Luego la expresión está conformada por varias subexpresiones, estas se resuelven según las reglas de precedencia de su operador (se utilizará la letra S para indicar cada subexpresión):



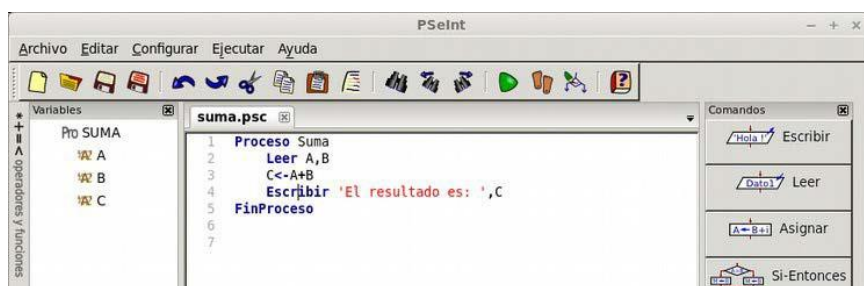
1. Se resuelve primero la potenciación $S1 \leftarrow t^2$, con lo cual S1 vale 100.
2. Luego hay un conjunto de productos y divisiones en el mismo nivel, por lo tanto tendrá prioridad la de la izquierda: $S2 \leftarrow v1 * t$, $(100 * 10)$ resultado 1000.
3. $S3 \leftarrow 1 / 2$, resultado 0.5
4. $S4 \leftarrow S3 * g$, $(0.5 * 9.8)$ resultado 4.9
5. $S5 \leftarrow S4 * S1$, $(4.9 * 100)$ resultado 490
6. finalmente la resta que posee el menor nivel de precedencia, $S6 \leftarrow S2 - S5$, $(1000 - 490)$ resultado final 510.

Observemos que nos aparece parte de un pseudocódigo, entre las sentencias Proceso y FinProceso deberemos escribir nuestro código, pero mejor será utilizar los botones que se ven al costado derecho en la barra cuyo nombre es Comandos.

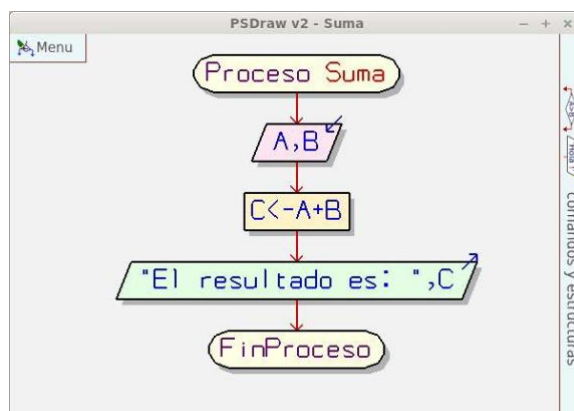
Lo primero que debemos poner es un ingreso de datos, por lo tanto debemos utilizar el botón Leer. Ahora debemos reemplazar el recuadro que dice “Lista de Variables” por nuestras variables separadas por comas, es lo que nos sugiere en el cuadro inferior que apareció al presionar el botón.

Como vemos, está toda la línea grisada, para seleccionar solamente el cuadro que dice “Lista de Variables” nos movemos con las flechas de cursor hacia atrás y luego hacia adelante hasta que se seleccione solamente lo que queremos. Una vez seleccionado el cuadro de lista debemos tipear nuestras variables según el algoritmo diseñado. Repetimos las acciones con los botones de asignar y Escribir con lo que debemos obtener el pseudocódigo completo. Podemos también cambiar el nombre del proceso, le pondremos de nombre “Suma”.

Antes de continuar es conveniente grabar el pseudocódigo generado, presionamos el ícono y nos aparecerá un cuadro de diálogo preguntando el nombre del archivo y donde guardarlo. Una vez hecho esto, la ventana quedará como la vemos en la siguiente imagen:



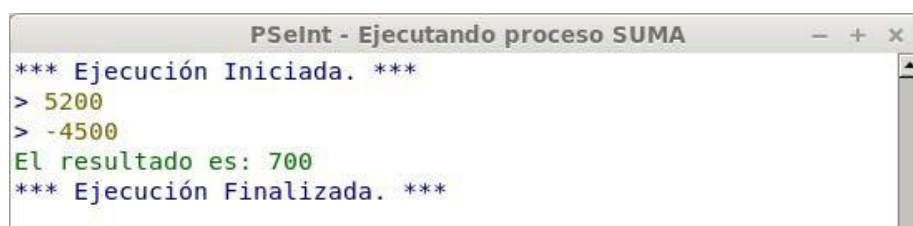
Ahora podemos cambiar a vista de Diagrama de Flujo, simplemente presionando el botón con el ícono en la barra de botones nos abrirá una ventana donde podremos apreciar como queda el diagrama de flujo de datos, también lo podremos exportar como imagen:



Ahora que el algoritmo está terminado, probaremos mediante PseInt el funcionamiento del algoritmo al ser ejecutado por la computadora. Presionamos ahora el ícono ejecutar de la barra de botones y se nos abrirá una nueva ventana:



La idea es que ingresemos ambas variable que deberán ser sumadas, por ejemplo tomaremos como lote de prueba los valores 5200 y -4500, si el algoritmo está correcto el resultado debería ser 700... veamos: Ingresamos primer o el valor 5200 y presionamos intro (o enter), luego el valor -4500 y de nuevo Intro. Inmediatamente nos muestra el resultado que efectivamente era lo que esperábamos:



Digamos que este algoritmo fue diseñado solo con lo esencial para ver si funciona, pero como lo estamos ejecutando vemos que podría ser un poco más amigable, entonces reformulemos un poquito las instrucciones del pseudocódigo

```
1  Proceso Suma
2      Escribir "A continuación ingrese dos valores para sumarlos"
3      Leer A,B
4      C<-A+B
5      Escribir 'El resultado es: ',C
6  FinProceso
```

ALGORITMO DEL TRIANGULO

Veamos como encarar un problema un poco más complejo. El enunciado sería: *Calcular el área y el perímetro de un triángulo conociendo el largo de los 3 lados.*

Parece simple no? Bueno, si es simple, por lo menos para calcular perímetro = lado + lado + lado... Sí... pero hay un problema, seguramente recordamos que el área de un triángulo se calcula con la fórmula:

$$\text{área} = (\text{base} \times \text{altura}) / 2.$$

En el caso que los datos ingresados correspondan a un triángulo rectángulo, no habría problemas simplemente habría que identificar que lado corresponde a la base y cual a la altura, ¿pero si no es un triángulo rectángulo? Habría que calcular el valor altura... ¿y cómo se calcula? Bueno, podemos ir a un buscador de la web y consultar un sitio que nos diga “cómo se calcula el área de un triángulo escaleno”. El buscador nos llevaría a varios sitios donde encontraremos el “teorema de Herón de Alejandría” que dice: «En un

triángulo de lados a, b, c, y semiperímetro $s=(a+b+c)/2$, su área es igual a la raíz cuadrada de $s(s-a)(s-b)(s-c)$.» (sacado de la wikipedia). Bueno, el teorema habla de cualquier tipo de triángulo, justo es lo que necesitábamos. A esto nos referimos por comprensión del problema.

Entonces ya podemos definir las variables:

Variables de entrada: los 3 lados que llamaremos L1, L2 y L3

Variables de salida: el área que la llamaremos A y el perímetro que lo llamaremos P.

Variables de proceso: Según el teorema de Herón, necesitamos calcular el semiperímetro, al que llamaremos S. las fórmulas a aplicar serían:

$$P = L1 + L2 + L3 \quad S = P / 2$$

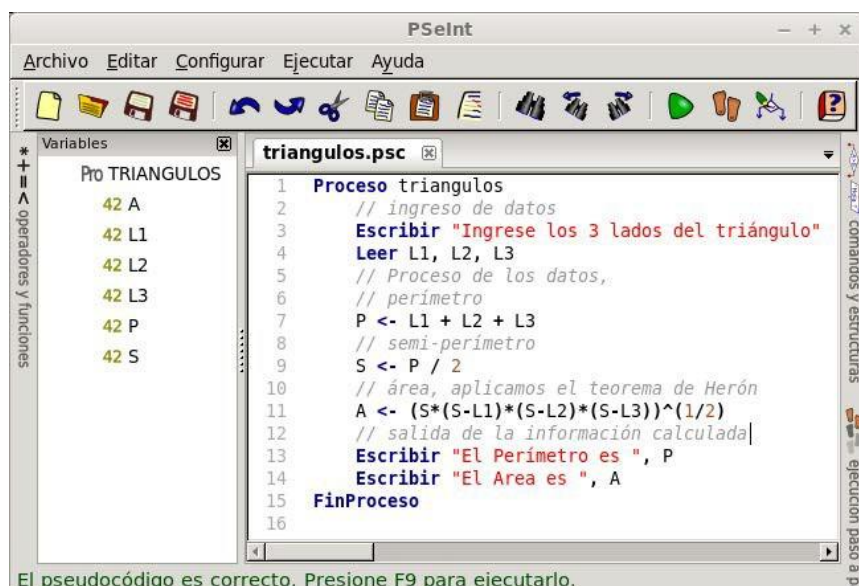
$$A = \sqrt{S \times (S - L1) \times (S - L2) \times (S - L3)}$$

Diseñemos entonces el algoritmo utilizando pseudocódigo y el DFD.

Pero primero haremos un par de observaciones:

Para obtener la raíz cuadrada recurrimos a las matemáticas, la raíz cuadrada de un número es equivalente a elevar ese número a $1/2$.

Por otra parte, agregaremos comentarios al pseudocódigo para recordar en un futuro de donde sacamos la fórmula. El programa PseInt admite el añadido de comentarios al pseudocódigo utilizando una doble barra (//) al principio del texto. Esta forma de comentar es la que se utiliza en el lenguaje de programación "C++" y en muchos de los lenguajes basados en este, como Java, PHP, Javascript, etc.



The screenshot shows the PSeInt application window. The title bar says "PSeInt". The menu bar includes "Archivo", "Editar", "Configurar", "Ejecutar", and "Ayuda". The toolbar contains various icons for file operations and execution. On the left, there is a "Variables" panel with a tree view showing "Pro TRIANGULOS" and a list of variables: A, L1, L2, L3, P, and S, each with a value of 42. The main editor displays a pseudocode program in "triangulos.psc". The code includes comments in Spanish explaining each step: input of data, calculation of perimeter (P), semi-perimeter (S), and the area (A) using Heron's formula. The program ends with "FinProceso". At the bottom, a status bar message reads: "El pseudocódigo es correcto. Presione F9 para ejecutarlo."

```
1  Proceso triangulos
2  // ingreso de datos
3  Escribir "Ingrese los 3 lados del triángulo"
4  Leer L1, L2, L3
5  // Proceso de los datos,
6  // perímetro
7  P <- L1 + L2 + L3
8  // semi-perímetro
9  S <- P / 2
10 // área, aplicamos el teorema de Herón
11 A <- (S*(S-L1)*(S-L2)*(S-L3))^(1/2)
12 // salida de la información calculada
13 Escribir "El Perímetro es ", P
14 Escribir "El Area es ", A
15 FinProceso
16
```

Y el diagrama quedaría de la siguiente forma

